AI Application Development and JIRA Practicals

Complete step-by-step solutions to all 44 questions

Practicals 1-44	Python code with real program output (AI practicals)
Jira Cloud	Click-by-click steps (Jira practicals)
Tested	All Python programs were executed; the output shown is the actual output

How to run the code: install Python 3.9+, then `pip install scikit-learn pandas numpy pgmpy`. Save each program as a .py file (or paste into Jupyter / Google Colab) and run it.

Note: Jira's menus are updated often, so a label may differ slightly in your version. Sign in at your-site.atlassian.net (free plan: atlassian.com/software/jira/free).

Note: Questions 24/25 and 31/32 are repeated in the question bank, and several Jira questions overlap (6/34, 8/36, 10/38, 12/28, 14/30, 20/40, 22/44). Each still has its own steps below.

Contents

1. PEAS, environment classification and table-driven vacuum-cleaner agent 4
2. Create a Jira Cloud Kanban project and explore the dashboard 5
3. PEAS for an autonomous taxi and task-environment classification 6
4. Create a Jira project, add team members and assign project roles 6
5. Breadth-First Search (BFS) for the 8-puzzle 7
6. Configure issue types and create components and versions 8
7. Water-jug problem (4 L and 3 L) using Depth-First Search 9
8. Create an Epic with related issues, assignee, priority and due date 10
9. Uniform Cost Search (UCS): Arad to Bucharest 10
10. Design a custom workflow and associate it with a project 11
11. UCS on a user-defined weighted graph 11
12. Create and configure a Scrum board, run a sprint, generate a Sprint Report 12
13. Greedy Best-First Search for the 8-puzzle (Manhattan distance) 12
14. Company-managed Kanban project with board configuration and Control Chart 14
15. A* search for the 8-puzzle (Manhattan distance) 14
16. JQL queries by status, assignee and priority; save as filters 16
17. A* search: Arad to Bucharest with straight-line distance 16
18. Generate a Burndown Chart and Velocity Chart 17
19. Bayes' Rule: probability of disease after a positive test 17
20. Create a custom dashboard with gadgets 18
21. Naive Bayes classifier with scikit-learn (spam dataset) 18
22. View a permission scheme, configure project roles and assign permissions 19
23. Burglary-Alarm Bayesian Network with pgmpy 19
24. Jira Automation rule: notify or auto-assign when an issue moves to Done 20
25. Jira Automation rule (repeat of Practical 24) 20
26. Jira mobile app: view, update status, comment and assign 21
27. Linear Regression with scikit-learn 21
28. Create a Scrum project and board, add issues to a sprint, start and review progress 22
29. Binary classifier: digit '5' versus not '5' 22
30. Kanban board: columns, move issues and generate a Control Chart 23
31. k-fold cross-validation (scikit-learn) 23
32. k-fold cross-validation (repeat of Practical 31) 24
33. Decision Tree classifier on the Iris dataset 24
34. Create and configure issue types, components and versions 25
35. Decision Tree Regression 25
36. Epic with Stories and Tasks, assigned with priority and due date 26
37. k-Nearest Neighbors (kNN) classifier 26

38. Customize a workflow by adding statuses and transitions 27
39. Multiclass classification with Logistic Regression 27
40. Pie Chart / Issue Statistics report on a custom dashboard 28
41. Class imbalance and possible bias in a public dataset 28
42. Create, test and enable a Jira Automation rule 29
43. Naive Bayes with train / test split shown 29
44. Configure project roles and review role permissions 30

Practical 1: PEAS, environment classification and table-driven vacuum-cleaner agent

Aim: To define PEAS for a vacuum-cleaner agent, classify its environment and implement a simple table-driven agent in Python.

PEAS description

Component	Description
Performance measure	Amount of dirt cleaned, time taken, electricity used, noise produced, number of squares cleaned
Environment	Rooms / squares (A, B), floor, dirt, furniture, walls
Actuators	Wheels (Left, Right), suction brush, dust bag
Sensors	Dirt sensor, location sensor (GPS / bump sensor), wheel odometer

Task environment classification

Property	Type	Reason
Observable	Partially observable	It senses only the square it is in and cannot see dirt in other squares.
Deterministic?	Deterministic (in the basic model)	Sucking always cleans the square; next state is fully decided by action. Stochastic if dirt can reappear randomly.
Episodic?	Sequential	Current action (e.g. moving away) affects later percepts and future cleaning.
Static?	Static (basic) / Dynamic	World does not change while agent thinks, unless dirt appears on its own.
Discrete?	Discrete	Finite squares and actions.
Agents	Single agent	Only the vacuum cleaner acts.

Procedure (step by step)

1. Define the percepts: (location, status) where location is A or B, status is Clean or Dirty.

2. Define the actions: Suck, Right, Left, NoOp.

3. Create a table that maps every percept SEQUENCE to an action (generated with itertools.product).

4. Create the agent class: it stores the percept history and looks up tuple(history) in the table.

5. Create a small environment (two dirty squares) and run the agent for 3 steps.

6. Observe that the table grows exponentially (84 entries for just 3 steps), which is the main drawback of table-driven agents.

Python program

```
from itertools import product

# Percepts are (location, status). Actions: Suck, Right, Left, NoOp
percepts_all = [('A', 'Clean'), ('A', 'Dirty'), ('B', 'Clean'), ('B', 'Dirty')]

def rule(p):
    loc, status = p
    if status == 'Dirty':
        return 'Suck'
    return 'Right' if loc == 'A' else 'Left'

# Build the lookup TABLE: every percept sequence (length 1..3) -> action
table = {}
for length in range(1, 4):
    for seq in product(percepts_all, repeat=length):
        table[seq] = rule(seq[-1])
print("Table entries:", len(table))

class TableDrivenAgent:
    def __init__(self, table):
        self.table = table
        self.percepts = []            # percept history
    def act(self, percept):
```

```
        self.percepts.append(percept)
        return self.table.get(tuple(self.percepts), 'NoOp')

# Simple environment: two squares A and B, both dirty, agent starts at A
world = {'A': 'Dirty', 'B': 'Dirty'}
loc = 'A'
agent = TableDrivenAgent(table)

for step in range(1, 4):
    percept = (loc, world[loc])
    action = agent.act(percept)
    print(f"Step {step}: percept={percept} -> action={action}")
    if action == 'Suck':
        world[loc] = 'Clean'
    elif action == 'Right':
        loc = 'B'
    elif action == 'Left':
        loc = 'A'
print("Final world:", world)
```

Output

```
Table entries: 84
Step 1: percept=('A', 'Dirty') -> action=Suck
Step 2: percept=('A', 'Clean') -> action=Right
Step 3: percept=('B', 'Dirty') -> action=Suck
Final world: {'A': 'Clean', 'B': 'Clean'}
```

Practical 2: Create a Jira Cloud Kanban project and explore the dashboard

Aim: To create a Kanban project, configure project details and explore the dashboard.

Steps in Jira

1. Sign in to Jira Cloud. Click Projects in the top bar, then Create project.

2. Choose Software development, then select the Kanban template and click Use template.

3. Choose the project type: Team-managed (simple, per-project settings) or Company-managed (shared schemes). Pick Team-managed for a quick start.

4. Enter the Name (for example: AI Lab Project) and Key (for example: AIL). The key becomes the prefix of every issue (AIL-1).

5. Set Access (Open / Limited / Private) and click Create.

6. Configure details: open the project, click Project settings (bottom of left sidebar) > Details. Edit name, key, description, avatar and project lead.

7. Explore: Summary (overview and activity), Timeline / Backlog, Board (columns To Do, In Progress, Done), List, Calendar, Reports and Project settings.

8. Create the first issue with Create (+) at the top and watch it appear in To Do on the board.

Screenshot: project Summary page, Details page, and the board.

Practical 3: PEAS for an autonomous taxi and task-environment classification

Aim: To define PEAS for a self-driving taxi and classify its task environment with examples.

PEAS description

Component	Description
Performance measure	Safety, reaching destination, minimum time and fuel, passenger comfort, obeying traffic laws, profit
Environment	Roads, traffic, pedestrians, cyclists, traffic signals, weather, customers
Actuators	Steering, accelerator, brake, indicators, horn, display / speaker for passengers
Sensors	Cameras, LiDAR, radar, GPS, speedometer, odometer, accelerometer, engine sensors, microphone

Task environment classification

Property	Type	Reason
Observable	Partially observable	Cannot see around corners or know other drivers' intentions.
Deterministic?	Stochastic	Other cars, pedestrians and road conditions are unpredictable.
Episodic?	Sequential	Turning left now affects the whole later route and safety.
Static?	Dynamic	Traffic keeps changing while the taxi decides.
Discrete?	Continuous	Speed, position and steering angle take continuous values.
Agents	Multi-agent	Many other vehicles and people, cooperative (avoid collision) and competitive (overtaking).

Procedure (step by step)

1. Write the PEAS table (Performance, Environment, Actuators, Sensors).
2. For each environment property, decide the value and write a one-line reason with a taxi example.
3. Example: a pedestrian suddenly crossing makes the environment stochastic and dynamic.
4. Conclude: an autonomous taxi works in one of the hardest environments - partially observable, stochastic, sequential, dynamic, continuous and multi-agent.

Practical 4: Create a Jira project, add team members and assign project roles

Aim: To add users to a project and give them suitable roles.

Steps in Jira

1. Create a project (see Practical 2).
2. Open the project, go to Project settings > People (team-managed: Access). Click Add people.
3. Type the name or email of the person. Invite users who do not yet have an account.
4. Choose a role: Administrator (manage the project), Member (create and edit issues), Viewer (read only) in team-managed projects. In company-managed projects the default roles are Administrator, Developers, Users and Viewers.
5. Click Add. Repeat for each team member (for example: Project Manager = Administrator, Developers = Member, Client = Viewer).
6. To change a role later, use the role drop-down next to the user in the People list, or choose Remove access.
7. Verify by opening the People page and checking each user and role.

Practical 5: Breadth-First Search (BFS) for the 8-puzzle

Aim: To implement BFS in Python to find the shortest solution of the 8-puzzle from a given initial state to a goal state.

Procedure (step by step)

1. Represent a state as a tuple of 9 numbers (0 = blank).

2. Write neighbors(state): find the blank and generate states obtained by sliding Up, Down, Left, Right.

3. Keep a FIFO queue (deque) of states and a parent dictionary (also serves as the visited set).

4. Pop the oldest state; if it equals the goal, trace back through parent to build the path.

5. Otherwise push every unseen neighbour to the queue.

6. Print the path. BFS explores level by level, so the first solution found has the fewest moves (optimal when each move costs 1).

7. Replace START and GOAL with the configuration given in your exam paper.

Python program

```
GOAL  = (1, 2, 3, 8, 0, 4, 7, 6, 5)      # 0 = blank
START = (2, 8, 3, 1, 6, 4, 7, 0, 5)      # change to your given configuration

def neighbors(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    for name, dr, dc in (('Up', -1, 0), ('Down', 1, 0), ('Left', 0, -1), ('Right', 0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            s = list(state)
            s[i], s[j] = s[j], s[i]
            yield name, tuple(s)

def show(state):
    for k in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[k:k + 3]))
    print()

def manhattan(state, goal=GOAL):
    d = 0
    for tile in range(1, 9):
        i, j = state.index(tile), goal.index(tile)
        d += abs(i // 3 - j // 3) + abs(i % 3 - j % 3)
    return d

def build_path(parent, state):
    path = []
    while parent[state][0] is not None:
        prev, move = parent[state]
        path.append((move, state))
        state = prev
    return path[::-1]

from collections import deque

def bfs(start, goal):
    queue = deque([start])
    parent = {start: (None, None)}
    while queue:
        state = queue.popleft()
        if state == goal:
            return build_path(parent, state), len(parent)
        for move, nxt in neighbors(state):
            if nxt not in parent:
                parent[nxt] = (state, move)
                queue.append(nxt)
    return None, len(parent)

path, explored = bfs(START, GOAL)
print("Initial state:")
show(START)
for i, (move, state) in enumerate(path, 1):
    print(f"Move {i}: {move}")
```

```
    show(state)
print("Shortest solution length:", len(path), "moves")
print("States generated:", explored)
```

Output

```
Initial state:
2 8 3
1 6 4
7 _ 5

Move 1: Up
2 8 3
1 _ 4
7 6 5

Move 2: Up
2 _ 3
1 8 4
7 6 5

Move 3: Left
_ 2 3
1 8 4
7 6 5

Move 4: Down
1 2 3
_ 8 4
7 6 5

Move 5: Right
1 2 3
8 _ 4
7 6 5

Shortest solution length: 5 moves
States generated: 62
```

Practical 6: Configure issue types and create components and versions

Aim: To configure Epic, Story, Task and Bug issue types, and create components and versions.

Steps in Jira

1. Open Project settings > Issue types (team-managed) or Issue types > Issue type scheme (company-managed).

2. Make sure Epic, Story, Task and Bug are present. Use Add issue type to add missing ones and choose an icon and description.

3. Optionally add custom fields to each issue type (Fields > Add field) and drag fields into the layout.

4. Components (company-managed): Project settings > Components > Create component. Enter name (for example: Frontend, Backend, Database), description, component lead and default assignee. Click Save.

5. Versions (company-managed): Project settings > Versions > Add version. Enter name (v1.0), start date, release date and description. Click Add.

6. For team-managed projects, versions and components are under Project settings > Features (turn on Releases / Components) or in the Releases menu.

7. Create a test issue and set its Component and Fix version fields to confirm they work.

Practical 7: Water-jug problem (4 L and 3 L) using Depth-First Search

Aim: To obtain exactly 2 litres using a 4-litre and a 3-litre jug with DFS, and display the sequence of states.

Procedure (step by step)

1. State = (x, y) where x is water in the 4 L jug and y in the 3 L jug. Start = (0, 0). Goal: x = 2.

2. Operators: fill 4 L, fill 3 L, empty 4 L, empty 3 L, pour 4 L into 3 L, pour 3 L into 4 L.

3. Use a stack (LIFO) holding (state, path so far) and a visited set.

4. Pop a state, skip it if visited, stop if goal, else push all unvisited successors.

5. Print the path. DFS finds a valid solution but not necessarily the shortest one. If the goal is relaxed to '2 L in either jug', the 4-step path (0,0) -> (0,3) -> (3,0) -> (3,3) -> (4,2) also works.

Python program

```
CAP_X, CAP_Y, TARGET = 4, 3, 2

def successors(state):
    x, y = state
    yield "Fill 4L jug",           (CAP_X, y)
    yield "Fill 3L jug",           (x, CAP_Y)
    yield "Empty 4L jug",          (0, y)
    yield "Empty 3L jug",          (x, 0)
    t = min(x, CAP_Y - y)
    yield "Pour 4L -> 3L",         (x - t, y + t)
    t = min(y, CAP_X - x)
    yield "Pour 3L -> 4L",         (x + t, y - t)

def dfs(start=(0, 0)):
    stack = [(start, [("Start", start)])]
    visited = set()
    while stack:
        state, path = stack.pop()
        if state in visited:
            continue
        visited.add(state)
        if state[0] == TARGET:          # exactly 2 litres in the 4-litre jug
            return path
        for action, nxt in successors(state):
            if nxt not in visited:
                stack.append((nxt, path + [(action, nxt)]))
    return None

solution = dfs()
for step, (action, state) in enumerate(solution):
    print(f"{step:2d}. {action:<15} -> (4L: {state[0]}, 3L: {state[1]})")
print("Goal reached: 2 litres in the 4-litre jug")
```

Output

```
 0. Start           -> (4L: 0, 3L: 0)
 1. Fill 3L jug     -> (4L: 0, 3L: 3)
 2. Pour 3L -> 4L   -> (4L: 3, 3L: 0)
 3. Fill 3L jug     -> (4L: 3, 3L: 3)
 4. Pour 3L -> 4L   -> (4L: 4, 3L: 2)
 5. Empty 3L jug    -> (4L: 4, 3L: 0)
 6. Pour 4L -> 3L   -> (4L: 1, 3L: 3)
 7. Empty 3L jug    -> (4L: 1, 3L: 0)
 8. Pour 4L -> 3L   -> (4L: 0, 3L: 1)
 9. Fill 4L jug     -> (4L: 4, 3L: 1)
10. Pour 4L -> 3L   -> (4L: 2, 3L: 3)
Goal reached: 2 litres in the 4-litre jug
```

Result

Exactly 2 litres is obtained in the 4-litre jug; the sequence of states is displayed above.

Practical 8: Create an Epic with related issues, assignee, priority and due date

Aim: To create an Epic, add child issues, assign users and set priority and due dates.

Steps in Jira

1. Open the Backlog or Timeline and click Create. Choose Issue type = Epic, enter a summary (for example: User Login Module) and create it.
2. Open the Epic. In the Child issues section, click Add child issue > Create child issue. Select Story or Task and give a summary (Design login page, Write authentication API).
3. Open each child issue. Set Assignee from the drop-down.
4. Set Priority (Highest, High, Medium, Low, Lowest).
5. Set Due date with the calendar field.
6. Return to the Epic to check the progress bar showing the percentage of child issues done.
7. Use the Timeline view to see the Epic and its issues on a schedule.

Practical 9: Uniform Cost Search (UCS): Arad to Bucharest

Aim: To find the least-cost path from Arad to Bucharest in the weighted Romania road map using UCS.

Procedure (step by step)

1. Store the map as a dictionary of dictionaries: graph[city][neighbour] = road distance.
2. Use a priority queue (heapq) ordered by path cost g(n).
3. Pop the cheapest path. If its last city is Bucharest, return the cost and path.
4. Otherwise push every neighbour with cost = cost so far + road distance.
5. Keep best[node] to skip worse duplicate paths.
6. Expected answer: Arad - Sibiu - Rimnicu Vilcea - Pitesti - Bucharest with cost 418 (the Fagaras route costs 450).

Python program

```
import heapq

graph = {
  'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
  'Zerind': {'Arad': 75, 'Oradea': 71},
  'Oradea': {'Zerind': 71, 'Sibiu': 151},
  'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
  'Timisoara': {'Arad': 118, 'Lugoj': 111},
  'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
  'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
  'Drobeta': {'Mehadia': 75, 'Craiova': 120},
  'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
  'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
  'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
  'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
  'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
  'Giurgiu': {'Bucharest': 90},
  'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
  'Hirsova': {'Urziceni': 98, 'Eforie': 86},
  'Eforie': {'Hirsova': 86},
  'Vaslui': {'Urziceni': 142, 'Iasi': 92},
  'Iasi': {'Vaslui': 92, 'Neamt': 87},
  'Neamt': {'Iasi': 87},
}

def ucs(graph, start, goal):
    frontier = [(0, start, [start])]          # (cost so far, node, path)
    best = {}                                  # cheapest cost found per node
    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node == goal:
            return cost, path
```

```
        if node in best and best[node] <= cost:
            continue
        best[node] = cost
        for nbr, w in graph[node].items():
            heapq.heappush(frontier, (cost + w, nbr, path + [nbr]))
    return None, None

cost, path = ucs(graph, 'Arad', 'Bucharest')
print("Least-cost path:", " -> ".join(path))
print("Total cost     :", cost)
```

Output

```
Least-cost path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total cost     : 418
```

Practical 10: Design a custom workflow and associate it with a project

Aim: To create a workflow with statuses and transitions and attach it to a project.

Steps in Jira

1. Team-managed: Project settings > Workflow. Company-managed: Jira settings (gear) > Issues > Workflows > Add workflow.
2. Name the workflow (for example: Dev Flow).
3. Add statuses: Backlog, To Do, In Progress, Code Review, Testing, Done. Each status has a category (To Do, In Progress, Done).
4. Add transitions by dragging between statuses (Start work: To Do -> In Progress; Submit for review: In Progress -> Code Review; Pass: Testing -> Done; Reject: Testing -> In Progress).
5. Optionally add conditions, validators and post functions on a transition (Transition > Rules).
6. Click Publish (Update). In company-managed Jira, go to Workflow schemes, create a scheme, map the issue types to the workflow and click Associate on the project.
7. Open the project board and verify the new columns and transitions.

Practical 11: UCS on a user-defined weighted graph

Aim: To apply UCS to your own weighted graph and print the least-cost path and its total cost.

Procedure (step by step)

1. Write the edges as (node1, node2, weight) tuples.
2. Convert the list to an adjacency dictionary (both directions for an undirected graph).
3. Apply the same UCS function as in Practical 9 with a heap ordered by total cost.
4. Choose a start and goal, run, and print path and cost.
5. Verify by hand: A-C (2) + C-B (1) + B-D (5) + D-E (2) + E-F (3) = 13.

Python program

```
import heapq

# User-defined weighted graph (undirected)
edges = [('A', 'B', 4), ('A', 'C', 2), ('B', 'C', 1), ('B', 'D', 5),
         ('C', 'D', 8), ('C', 'E', 10), ('D', 'E', 2), ('D', 'F', 6), ('E', 'F', 3)]
graph = {}
for u, v, w in edges:
    graph.setdefault(u, {})[v] = w
    graph.setdefault(v, {})[u] = w

def ucs(graph, start, goal):
    frontier = [(0, start, [start])]
    best = {}
    while frontier:
        cost, node, path = heapq.heappop(frontier)
```

```
        if node == goal:
            return cost, path
        if node in best and best[node] <= cost:
            continue
        best[node] = cost
        for nbr, w in graph[node].items():
            heapq.heappush(frontier, (cost + w, nbr, path + [nbr]))
    return None, None

start, goal = 'A', 'F'
cost, path = ucs(graph, start, goal)
print(f"Start = {start}, Goal = {goal}")
print("Least-cost path:", " -> ".join(path))
print("Total cost     :", cost)
```

Output

```
Start = A, Goal = F
Least-cost path: A -> C -> B -> D -> E -> F
Total cost     : 13
```

Practical 12: Create and configure a Scrum board, run a sprint, generate a Sprint Report

Aim: To set up a Scrum project, create and start a sprint and view the Sprint Report.

Steps in Jira

1. Create project > Software development > Scrum template. Name it and click Create.

2. Open Backlog and create 6-8 issues with story points (estimate) and assignees.

3. Click Create sprint, then drag issues from the backlog into the sprint.

4. Click Start sprint. Enter the sprint name, duration (1-4 weeks), start / end date and goal. Click Start.

5. On the Active sprint board move issues across the columns To Do > In Progress > Done during the sprint.

6. Click Complete sprint at the end; choose where unfinished issues go (backlog or next sprint).

7. Open Reports > Sprint report to see completed issues, issues not completed, scope change and a burndown line.

Practical 13: Greedy Best-First Search for the 8-puzzle (Manhattan distance)

Aim: To solve the 8-puzzle using Greedy Best-First Search with the Manhattan distance heuristic.

Procedure (step by step)

1. Heuristic h(n) = sum over tiles 1-8 of |row difference| + |column difference| from the goal position.

2. Greedy search orders the frontier ONLY by h(n) (ignores the cost so far).

3. Use a priority queue with entries (h, tie-breaker, state).

4. Pop the state with the smallest h; stop at the goal; else push unseen neighbours with their h.

5. Print the path and the number of expanded nodes. Greedy is fast but not guaranteed optimal or complete in general.

Python program

```
GOAL  = (1, 2, 3, 8, 0, 4, 7, 6, 5)      # 0 = blank
START = (2, 8, 3, 1, 6, 4, 7, 0, 5)      # change to your given configuration

def neighbors(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    for name, dr, dc in (('Up', -1, 0), ('Down', 1, 0), ('Left', 0, -1), ('Right', 0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            s = list(state)
```

```
            s[i], s[j] = s[j], s[i]
            yield name, tuple(s)

def show(state):
    for k in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[k:k + 3]))
    print()

def manhattan(state, goal=GOAL):
    d = 0
    for tile in range(1, 9):
        i, j = state.index(tile), goal.index(tile)
        d += abs(i // 3 - j // 3) + abs(i % 3 - j % 3)
    return d

def build_path(parent, state):
    path = []
    while parent[state][0] is not None:
        prev, move = parent[state]
        path.append((move, state))
        state = prev
    return path[::-1]

import heapq, itertools

def greedy_best_first(start, goal):
    counter = itertools.count()               # tie-breaker for the heap
    frontier = [(manhattan(start), next(counter), start)]
    parent = {start: (None, None)}
    expanded = 0
    while frontier:
        h, _, state = heapq.heappop(frontier)
        expanded += 1
        if state == goal:
            return build_path(parent, state), expanded
        for move, nxt in neighbors(state):
            if nxt not in parent:
                parent[nxt] = (state, move)
                heapq.heappush(frontier, (manhattan(nxt), next(counter), nxt))
    return None, expanded

path, expanded = greedy_best_first(START, GOAL)
print("Initial state (h = %d):" % manhattan(START))
show(START)
for i, (move, state) in enumerate(path, 1):
    print(f"Move {i}: {move}   h = {manhattan(state)}")
    show(state)
print("Moves in solution :", len(path))
print("Nodes expanded    :", expanded)
```

Output

```
Initial state (h = 5):
2 8 3
1 6 4
7 _ 5

Move 1: Up   h = 4
2 8 3
1 _ 4
7 6 5

Move 2: Up   h = 3
2 _ 3
1 8 4
7 6 5

Move 3: Left   h = 2
_ 2 3
1 8 4
7 6 5

Move 4: Down   h = 1
1 2 3
_ 8 4
7 6 5

Move 5: Right   h = 0
1 2 3
```

```
8 _ 4
7 6 5

Moves in solution : 5
Nodes expanded    : 6
```

Practical 14: Company-managed Kanban project with board configuration and Control Chart

Aim: To create a company-managed Kanban project, configure its board and track progress with a Control Chart.

Steps in Jira

1. Projects > Create project > Kanban > Use template > choose Company-managed. Fill in name and key.
2. Open the board. Click ... > Board settings > Columns. Rename or add columns (Backlog, Selected, In Progress, Review, Done) and map statuses to columns.
3. Set a WIP limit (Constraints) per column, for example Max 3 for In Progress.
4. Create at least 8-10 issues and move them through the columns over time so that cycle-time data exists.
5. Open Reports > Control Chart.
6. Select the board, columns and period. Read the chart: each dot is an issue, the blue line is the rolling average and the shaded band shows standard deviation of cycle / lead time.
7. Use it to find slow outliers and predict delivery time.

Practical 15: A* search for the 8-puzzle (Manhattan distance)

Aim: To implement A* search for the 8-puzzle, displaying the solution path and number of moves.

Procedure (step by step)

1. A* uses f(n) = g(n) + h(n), where g = moves made so far and h = Manhattan distance.
2. Manhattan distance is admissible and consistent, so A* returns an optimal (shortest) solution.
3. Use a heap of (f, g, tie-breaker, state) and a g_cost dictionary.
4. Update a state only when a cheaper g is found.
5. Print each move with g, h and f, then the total number of moves.

Python program

```
GOAL  = (1, 2, 3, 8, 0, 4, 7, 6, 5)      # 0 = blank
START = (2, 8, 3, 1, 6, 4, 7, 0, 5)      # change to your given configuration

def neighbors(state):
    i = state.index(0)
    r, c = divmod(i, 3)
    for name, dr, dc in (('Up', -1, 0), ('Down', 1, 0), ('Left', 0, -1), ('Right', 0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            s = list(state)
            s[i], s[j] = s[j], s[i]
            yield name, tuple(s)

def show(state):
    for k in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[k:k + 3]))
    print()

def manhattan(state, goal=GOAL):
    d = 0
    for tile in range(1, 9):
        i, j = state.index(tile), goal.index(tile)
        d += abs(i // 3 - j // 3) + abs(i % 3 - j % 3)
    return d

def build_path(parent, state):
```

```
    path = []
    while parent[state][0] is not None:
        prev, move = parent[state]
        path.append((move, state))
        state = prev
    return path[::-1]

import heapq, itertools

def astar(start, goal):
    counter = itertools.count()
    frontier = [(manhattan(start), 0, next(counter), start)]   # (f, g, tie, state)
    parent = {start: (None, None)}
    g_cost = {start: 0}
    expanded = 0
    while frontier:
        f, g, _, state = heapq.heappop(frontier)
        if g > g_cost[state]:
            continue
        expanded += 1
        if state == goal:
            return build_path(parent, state), expanded
        for move, nxt in neighbors(state):
            ng = g + 1
            if nxt not in g_cost or ng < g_cost[nxt]:
                g_cost[nxt] = ng
                parent[nxt] = (state, move)
                heapq.heappush(frontier, (ng + manhattan(nxt), ng, next(counter), nxt))
    return None, expanded

path, expanded = astar(START, GOAL)
print("Initial state:")
show(START)
for i, (move, state) in enumerate(path, 1):
    print(f"Move {i}: {move}   g = {i}, h = {manhattan(state)}, f = {i + manhattan(state)}")
    show(state)
print("Number of moves :", len(path))
print("Nodes expanded  :", expanded)
```

Output

```
Initial state:
2 8 3
1 6 4
7 _ 5

Move 1: Up   g = 1, h = 4, f = 5
2 8 3
1 _ 4
7 6 5

Move 2: Up   g = 2, h = 3, f = 5
2 _ 3
1 8 4
7 6 5

Move 3: Left   g = 3, h = 2, f = 5
_ 2 3
1 8 4
7 6 5

Move 4: Down   g = 4, h = 1, f = 5
1 2 3
_ 8 4
7 6 5

Move 5: Right   g = 5, h = 0, f = 5
1 2 3
8 _ 4
7 6 5

Number of moves : 5
Nodes expanded  : 6
```

Practical 16: JQL queries by status, assignee and priority; save as filters

Aim: To search issues using JQL and save queries as filters.

Steps in Jira

1. Click Filters > Advanced issue search (switch to JQL mode).

2. By status:

```
project = AIL AND status = "In Progress"
```

3. By assignee:

```
project = AIL AND assignee = currentUser()   (or assignee = "Name")
```

4. By priority:

```
project = AIL AND priority in (High, Highest) ORDER BY created DESC
```

5. Combined:

```
project = AIL AND status != Done AND assignee = currentUser() AND priority = High
```

6. Other useful ones:

```
duedate < now() AND statusCategory != Done   and   created >= -7d
```

7. Click Search to run, then Save as, give the filter a name (for example: My High Priority Open Issues) and click Submit.

8. Open Filters > View all filters to find, share or subscribe to saved filters.

Practical 17: A* search: Arad to Bucharest with straight-line distance

Aim: To find the shortest path from Arad to Bucharest using A* with straight-line distance (SLD) as heuristic.

Procedure (step by step)

1. Keep the road map graph and a second table sld[city] = straight-line distance to Bucharest.

2. f(n) = g(n) + h(n) where g = road cost so far and h = SLD.

3. Pop the lowest f from the heap; stop when Bucharest is popped.

4. Expansion order: Arad (366) -> Sibiu (393) -> Rimnicu Vilcea (413) -> Fagaras (415) -> Pitesti (417) -> Bucharest (418).

5. Result: the same optimal path of cost 418, but A* expands fewer cities than UCS.

Python program

```
import heapq

graph = {
 'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
 'Zerind': {'Arad': 75, 'Oradea': 71},
 'Oradea': {'Zerind': 71, 'Sibiu': 151},
 'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
 'Timisoara': {'Arad': 118, 'Lugoj': 111},
 'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
 'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
 'Drobeta': {'Mehadia': 75, 'Craiova': 120},
 'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
 'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
 'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
 'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
 'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
 'Giurgiu': {'Bucharest': 90},
 'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
 'Hirsova': {'Urziceni': 98, 'Eforie': 86},
 'Eforie': {'Hirsova': 86},
 'Vaslui': {'Urziceni': 142, 'Iasi': 92},
 'Iasi': {'Vaslui': 92, 'Neamt': 87},
 'Neamt': {'Iasi': 87},
}
# Straight-line distance (km) to Bucharest = heuristic h(n)
```

```
sld = {'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242, 'Eforie': 161,
       'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151, 'Iasi': 226, 'Lugoj': 244,
       'Mehadia': 241, 'Neamt': 234, 'Oradea': 380, 'Pitesti': 100,
       'Rimnicu Vilcea': 193, 'Sibiu': 253, 'Timisoara': 329, 'Urziceni': 80,
       'Vaslui': 199, 'Zerind': 374}

def astar(graph, h, start, goal):
    frontier = [(h[start], 0, start, [start])]       # (f, g, node, path)
    best_g = {}
    while frontier:
        f, g, node, path = heapq.heappop(frontier)
        if node == goal:
            return g, path
        if node in best_g and best_g[node] <= g:
            continue
        best_g[node] = g
        for nbr, w in graph[node].items():
            heapq.heappush(frontier, (g + w + h[nbr], g + w, nbr, path + [nbr]))
    return None, None

cost, path = astar(graph, sld, 'Arad', 'Bucharest')
print("A* shortest path:", " -> ".join(path))
print("Total cost      :", cost)
```

Output

```
A* shortest path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total cost      : 418
```

Practical 18: Generate a Burndown Chart and Velocity Chart

Aim: To generate both reports and explain the information each shows.

Steps in Jira

1. Use a Scrum project with an active sprint that has estimated issues (Practical 12).

2. Burndown: Reports > Burndown chart (select the sprint and estimation statistic = story points).

3. Velocity: Reports > Velocity chart (needs at least one completed sprint).

4. Burndown chart explains: the horizontal axis is time in the sprint, the vertical axis is remaining work. The grey guideline is the ideal pace and the red line is actual remaining work. Above the guideline = behind schedule; below = ahead. It also shows scope changes.

5. Velocity chart explains: bars for each past sprint comparing Commitment (estimated work at start) with Completed work. The average of completed bars is the team's velocity, used to plan how much work fits in the next sprint.

6. Compare both: burndown tracks progress inside one sprint, velocity tracks capacity across many sprints.

Practical 19: Bayes' Rule: probability of disease after a positive test

Aim: To compute P(Disease | Positive) with Bayes' Rule from given probabilities.

Procedure (step by step)

1. Write Bayes' rule: P(D|+) = P(+|D) x P(D) / P(+).

2. Given: prevalence P(D) = 0.01, sensitivity P(+|D) = 0.99, false-positive rate P(+|not D) = 0.05.

3. Total probability: P(+) = 0.99 x 0.01 + 0.05 x 0.99 = 0.0594.

4. Posterior: P(D|+) = 0.0099 / 0.0594 = 0.1667 (about 16.7%).

5. Meaning: even with an accurate test, a positive result for a rare disease is mostly a false alarm (base-rate effect).

Python program

```
# Bayes' Rule: P(D|+) = P(+|D) * P(D) / P(+)
p_disease        = 0.01     # prevalence  P(D)
sensitivity      = 0.99     # P(+ | D)
false_pos_rate   = 0.05     # P(+ | not D)
```

```
p_no_disease = 1 - p_disease
p_positive   = sensitivity * p_disease + false_pos_rate * p_no_disease   # total probability
posterior    = (sensitivity * p_disease) / p_positive

print(f"P(Positive)           = {p_positive:.4f}")
print(f"P(Disease | Positive)  = {posterior:.4f}  ({posterior * 100:.2f}%)")
```

Output

```
P(Positive)           = 0.0594
P(Disease | Positive)  = 0.1667  (16.67%)
```

Practical 20: Create a custom dashboard with gadgets

Aim: To create a dashboard and add gadgets such as Issue Statistics and Pie Chart.

Steps in Jira

1. Click Dashboards > Create dashboard.

2. Enter a name (Project Overview) and description, set sharing, then click Create.

3. Click Add gadget. Choose Pie Chart, select a saved filter or project and a statistic type (Status, Assignee, Priority) then Save.

4. Add Issue Statistics, choose Statistic Type = Assignee and save.

5. Add more gadgets: Created vs Resolved Chart, Filter Results, Activity Stream.

6. Drag gadgets to arrange, change layout (2 columns), and click Done.

7. Share the dashboard with your team or set it as favourite.

Practical 21: Naive Bayes classifier with scikit-learn (spam dataset)

Aim: To train and test a Naive Bayes classifier on a small spam dataset and display its accuracy.

Procedure (step by step)

1. Create a small list of messages and labels (1 = spam, 0 = not spam).

2. Split into training and testing sets with train_test_split (stratified).

3. Convert text to word-count vectors with CountVectorizer (fit on training only).

4. Train MultinomialNB on the training vectors.

5. Predict on the test vectors and print accuracy and confusion matrix.

Python program

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

messages = [
 "win a free prize now", "free money click here", "claim your free lottery reward",
 "urgent offer win cash now", "cheap loans free approval", "you won a free gift card",
 "limited offer buy now cheap", "congratulations you win free cash",
 "meeting at 10 tomorrow", "please review the project report", "lunch with the team today",
 "can we reschedule the call", "notes from the class attached", "see you at the office",
 "assignment submission deadline friday", "happy birthday have a great day"]
labels = [1]*8 + [0]*8                      # 1 = spam, 0 = not spam

X_train, X_test, y_train, y_test = train_test_split(
    messages, labels, test_size=0.25, random_state=42, stratify=labels)

vec = CountVectorizer()
Xtr = vec.fit_transform(X_train)
Xte = vec.transform(X_test)

model = MultinomialNB()
model.fit(Xtr, y_train)
pred = model.predict(Xte)
print("Training samples:", len(X_train), "| Testing samples:", len(X_test))
```

```
for text, p in zip(X_test, pred):
    print(f"  {'SPAM' if p else 'HAM ':4} <- {text}")
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

Output

```
Training samples: 12 | Testing samples: 4
  HAM  <- can we reschedule the call
  SPAM <- free money click here
  HAM  <- assignment submission deadline friday
  SPAM <- limited offer buy now cheap
Accuracy: 1.0
Confusion matrix:
 [[2 0]
 [0 2]]
```

Practical 22: View a permission scheme, configure project roles and assign permissions

Aim: To view a permission scheme, configure roles and grant suitable permissions.

Steps in Jira

1. Sign in as Jira administrator. Go to Project settings > Permissions to view the scheme used by the project.

2. Open Jira settings (gear) > Issues > Permission schemes and click the scheme name.

3. Review permissions such as Browse Projects, Create Issues, Edit Issues, Assign Issues, Transition Issues, Delete Issues and Administer Projects.

4. Click Edit next to a permission and choose who gets it: Project role (Administrators, Developers, Users), Group, Single user or Application role.

5. Configure roles: Jira settings > System > Project roles > add a role (for example: QA Tester) and assign users in Project settings > People.

6. Grant: Developers = Create, Edit, Assign, Transition; Users = Browse, Comment; Administrators = Administer Projects.

7. Use Permission helper (Project settings > Permissions) to test why a user can or cannot perform an action.

Practical 23: Burglary-Alarm Bayesian Network with pgmpy

Aim: To build the Burglary-Alarm network and compute P(Burglary | Alarm = True).

Procedure (step by step)

1. Install the library: pip install pgmpy.

2. Define the structure: Burglary -> Alarm and Earthquake -> Alarm.

3. Create CPDs: P(B=T) = 0.001, P(E=T) = 0.002, P(A=T | B,E) = 0.95 (T,T), 0.94 (T,F), 0.29 (F,T), 0.001 (F,F).

4. Add the CPDs to the model and call check_model().

5. Run exact inference with VariableElimination and query Burglary with evidence Alarm = 1.

6. Result: about 0.3736. The alarm raises the burglary probability from 0.1% to about 37%.

7. In older pgmpy versions, use BayesianNetwork instead of DiscreteBayesianNetwork.

Python program

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Structure: Burglary -> Alarm <- Earthquake
model = DiscreteBayesianNetwork([('Burglary', 'Alarm'), ('Earthquake', 'Alarm')])

cpd_b = TabularCPD('Burglary',   2, [[0.999], [0.001]])     # row0 = False, row1 = True
cpd_e = TabularCPD('Earthquake', 2, [[0.998], [0.002]])
cpd_a = TabularCPD('Alarm', 2,
```

```
        values=[[0.999, 0.71, 0.06, 0.05],                  # P(Alarm=False | B,E)
                [0.001, 0.29, 0.94, 0.95]],                  # P(Alarm=True  | B,E)
        evidence=['Burglary', 'Earthquake'], evidence_card=[2, 2])

model.add_cpds(cpd_b, cpd_e, cpd_a)
print("Model valid:", model.check_model())

infer = VariableElimination(model)
result = infer.query(variables=['Burglary'], evidence={'Alarm': 1})
print(result)
print("P(Burglary=True | Alarm=True) = %.4f" % result.values[1])
```

Output

```
Model valid: True
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.6264 |
+-------------+-----------------+
| Burglary(1) |          0.3736 |
+-------------+-----------------+
P(Burglary=True | Alarm=True) = 0.3736
```

Practical 24: Jira Automation rule: notify or auto-assign when an issue moves to Done

Aim: To create an automation rule with a trigger, condition and action.

Steps in Jira

1. Open Project settings > Automation (or Jira settings > System > Global automation) and click Create rule.
2. Trigger: choose Issue transitioned, From = any, To = Done.
3. Condition: click Add component > New condition > Issue fields condition, for example Issue type equals Bug (or Priority equals High).
4. Action 1: Add component > New action > Send email (to Reporter) with subject 'Issue {{issue.key}} is Done'.
5. Action 2 (optional): Assign issue to the project lead or Add comment 'Work completed and verified'.
6. Give the rule a name, click Turn on rule.
7. Test by moving an issue to Done, then open Automation > Audit log to see the rule execution.

Practical 25: Jira Automation rule (repeat of Practical 24)

Aim: Same question as Practical 24 (it appears twice in the bank).

Steps in Jira

1. Follow steps 1-7 of Practical 24.
2. Alternative example: Trigger = Issue created, Condition = Priority is Highest, Action = Assign to team lead and send a Slack/email message.
3. Alternative example: Trigger = Scheduled (every Monday), Action = Create issue 'Weekly review'.

Practical 26: Jira mobile app: view, update status, comment and assign

Aim: To configure the Jira mobile app and manage an issue.

Steps in Jira

1. Install Jira Cloud from Google Play (Android) or App Store (iOS).

2. Open the app, tap Log in, enter your Atlassian account email and password, and choose your site.

3. Allow notifications and pick the projects you follow.

4. Tap a project or search and open an issue to see summary, description, status, assignee and priority.

5. Update status: tap the status button (To Do) and pick the new status (In Progress).

6. Add a comment: scroll to Comments, tap Add a comment, type the text and tap Send. Use @name to mention a colleague.

7. Assign: tap Assignee, choose the user, and confirm. The change shows instantly in the web version.

Practical 27: Linear Regression with scikit-learn

Aim: To implement Linear Regression, display predictions and an evaluation metric.

Procedure (step by step)

1. Take a simple dataset: years of experience (X) versus salary (y).

2. Split into training and test sets (75 / 25).

3. Fit LinearRegression; the model learns y = m x + c.

4. Predict the test values and print actual versus predicted.

5. Evaluate with Mean Squared Error and R2 score (R2 close to 1 means a good fit).

Python program

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Dataset: years of experience -> salary (in thousands)
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12]])
y = np.array([30, 35, 41, 44, 50, 55, 58, 65, 69, 74, 79, 83])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=1)

model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Slope (coef)  :", round(model.coef_[0], 3))
print("Intercept     :", round(model.intercept_, 3))
for x, actual, p in zip(X_test.ravel(), y_test, pred):
    print(f"Experience={x:2d}  Actual={actual}  Predicted={p:.2f}")
print("MSE :", round(mean_squared_error(y_test, pred), 3))
print("R2  :", round(r2_score(y_test, pred), 4))
```

Output

```
Slope (coef)  : 4.856
Intercept     : 25.276
Experience= 3  Actual=41  Predicted=39.84
Experience= 4  Actual=44  Predicted=44.70
Experience= 5  Actual=50  Predicted=49.56
MSE : 0.674
R2  : 0.9519
```

Practical 28: Create a Scrum project and board, add issues to a sprint, start and review progress

Aim: To set up a Scrum project, plan a sprint and review sprint progress.

Steps in Jira

1. Projects > Create project > Scrum template > create.

2. In Backlog, click Create issue and add 6-8 Stories / Tasks with story points.

3. Click Create sprint (Sprint 1) and drag the selected issues into it.

4. Click Start sprint and fill in the duration, dates and sprint goal.

5. Open the Active sprints board and drag cards across To Do > In Progress > Done daily.

6. Review progress with the Sprint burndown on the board, the Sprint report and the Burndown chart under Reports.

7. End with Complete sprint and discuss what was completed in a retrospective.

Practical 29: Binary classifier: digit '5' versus not '5'

Aim: To build a binary classifier on an MNIST-derived dataset and report accuracy.

Procedure (step by step)

1. Load scikit-learn's built-in digits dataset (1,797 images of 8x8 pixels, an MNIST-like set, no download needed).

2. Convert the 10 labels to binary: 1 if the digit is 5, otherwise 0.

3. Scale pixels (divide by 16) and split into training and testing sets (stratified).

4. Train LogisticRegression and predict.

5. Report accuracy and confusion matrix. Note: only about 10% of samples are '5', so accuracy alone can mislead; check precision and recall too.

6. For the full 70,000-image MNIST use fetch_openml('mnist_784') with the same steps.

Python program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

digits = load_digits()                  # 8x8 images of digits 0-9 (MNIST-like)
X, y = digits.data, digits.target

# Binary task: "is the digit 5?" (1) versus "not 5" (0)
y_bin = (y == 5).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X / 16.0, y_bin, test_size=0.25, random_state=42, stratify=y_bin)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)

print("Class counts (not-5, 5):", [int((y_bin == 0).sum()), int((y_bin == 1).sum())])
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

Output

```
Class counts (not-5, 5): [1615, 182]
Accuracy: 0.9933
Confusion matrix:
 [[403   1]
 [  2  44]]
```

Practical 30: Kanban board: columns, move issues and generate a Control Chart

Aim: To configure a Kanban board, move issues across statuses and generate a Control Chart.

Steps in Jira

1. Create a Kanban project (Practical 2 or 14) and open the board.

2. Board settings > Columns: add or rename columns and map each status to a column.

3. Add WIP limits to In Progress to prevent overload.

4. Create several issues and drag them across the columns (To Do > In Progress > Review > Done).

5. Leave time between moves (or use older sample data) so that cycle times differ.

6. Open Reports > Control Chart, choose the time window and columns, and read the average, rolling average and standard deviation.

Practical 31: k-fold cross-validation (scikit-learn)

Aim: To perform k-fold cross-validation on a classifier and display fold scores and mean accuracy.

Procedure (step by step)

1. Load the Iris dataset and choose a model (LogisticRegression).

2. Create KFold with 5 splits, shuffle = True.

3. Call cross_val_score; each fold trains on 4 parts and tests on the remaining part.

4. Print each fold score, the mean accuracy and the standard deviation.

Python program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)
model = LogisticRegression(max_iter=200)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')

for i, s in enumerate(scores, 1):
    print(f"Fold {i}: accuracy = {s:.4f}")
print("All scores   :", scores.round(4))
print("Mean accuracy: %.4f" % scores.mean())
print("Std deviation: %.4f" % scores.std())
```

Output

```
Fold 1: accuracy = 1.0000
Fold 2: accuracy = 1.0000
Fold 3: accuracy = 0.9333
Fold 4: accuracy = 0.9667
Fold 5: accuracy = 0.9667
All scores   : [1.     1.     0.9333 0.9667 0.9667]
Mean accuracy: 0.9733
Std deviation: 0.0249
```

Practical 32: k-fold cross-validation (repeat of Practical 31)

Aim: Same as Practical 31 (this question appears twice in the bank).

Procedure (step by step)

1. Follow exactly the steps and code of Practical 31.

2. Optional variation: use StratifiedKFold or change the model to DecisionTreeClassifier or KNeighborsClassifier.

Python program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)
model = LogisticRegression(max_iter=200)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')

for i, s in enumerate(scores, 1):
    print(f"Fold {i}: accuracy = {s:.4f}")
print("All scores   :", scores.round(4))
print("Mean accuracy: %.4f" % scores.mean())
print("Std deviation: %.4f" % scores.std())
```

Output

```
Fold 1: accuracy = 1.0000
Fold 2: accuracy = 1.0000
Fold 3: accuracy = 0.9333
Fold 4: accuracy = 0.9667
Fold 5: accuracy = 0.9667
All scores   : [1.     1.     0.9333 0.9667 0.9667]
Mean accuracy: 0.9733
Std deviation: 0.0249
```

Practical 33: Decision Tree classifier on the Iris dataset

Aim: To implement a Decision Tree classifier and display predictions and accuracy.

Procedure (step by step)

1. Load Iris (150 flowers, 4 features, 3 classes).

2. Split 70 / 30 with stratification.

3. Train DecisionTreeClassifier (Gini impurity, max_depth = 3).

4. Predict, compare with actual labels, and print accuracy and the classification report.

5. export_text prints the learned rules; petal length and width are the key features.

Python program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42, stratify=iris.target)

tree = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
tree.fit(X_train, y_train)
pred = tree.predict(X_test)

print("Predictions :", pred)
print("Actual      :", y_test)
print("Accuracy    :", round(accuracy_score(y_test, pred), 4))
print(classification_report(y_test, pred, target_names=iris.target_names))
print(export_text(tree, feature_names=list(iris.feature_names)))
```

Output

```
Predictions : [2 1 2 1 2 2 1 1 0 2 0 0 2 2 0 2 1 0 0 0 1 0 1 2 2 1 1 1 1 0 2 2 1 0 2 0 0
 0 0 2 1 0 2 2 1]
Actual      : [2 1 2 1 2 2 1 1 0 2 0 0 2 2 0 2 1 0 0 0 1 0 1 2 2 1 1 1 1 0 2 2 1 0 2 0 0
 0 0 1 1 0 2 2 1]
Accuracy    : 0.9778
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       1.00      0.93      0.97        15
   virginica       0.94      1.00      0.97        15

    accuracy                           0.98        45
   macro avg       0.98      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45

|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- class: 2
```

Practical 34: Create and configure issue types, components and versions

Aim: To configure issue types, components and versions in a Jira project.

Steps in Jira

1. Follow all steps of Practical 6.

2. Additionally create a new issue type 'Improvement' (Project settings > Issue types > Add issue type).

3. Create components: Frontend, Backend, Testing. Create versions: v1.0 and v1.1 with release dates.

4. Create an issue and choose Component and Fix Version; mark v1.0 as Released when finished (Releases > Release).

Practical 35: Decision Tree Regression

Aim: To implement Decision Tree Regression and display predictions with an evaluation metric.

Procedure (step by step)

1. Load the diabetes regression dataset (442 patients, 10 features, target = disease progression).

2. Split 80 / 20.

3. Train DecisionTreeRegressor with max_depth = 4 to avoid overfitting.

4. Predict the test set and print predictions next to actual values.

5. Evaluate with MSE, RMSE and R2 score.

Python program

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

reg = DecisionTreeRegressor(max_depth=4, random_state=42)
```

```
reg.fit(X_train, y_train)
pred = reg.predict(X_test)

print("First 8 predictions:", pred[:8].round(1))
print("First 8 actual     :", y_test[:8])
print("MSE :", round(mean_squared_error(y_test, pred), 2))
print("RMSE:", round(mean_squared_error(y_test, pred) ** 0.5, 2))
print("R2  :", round(r2_score(y_test, pred), 4))
```

Output

```
First 8 predictions: [175.3 182.8 175.3 241.2 121.9  90.3 203.5 241.2]
First 8 actual     : [219.  70. 202. 230. 111.  84. 242. 272.]
MSE : 3568.97
RMSE: 59.74
R2  : 0.3264
```

Practical 36: Epic with Stories and Tasks, assigned with priority and due date

Aim: To create an Epic with Stories and Tasks and set assignees, priority and due dates.

Steps in Jira

1. Create the Epic 'Online Shopping App' (Create > Issue type Epic).

2. Inside the Epic add Stories: 'As a user I can log in', 'As a user I can add to cart'.

3. Add Tasks under each Story as sub-tasks or child issues (Create database table, Design UI).

4. For every issue set Assignee, Priority and Due date.

5. Open the Epic or Timeline and confirm progress and dates. Use the filter Epic Link to list all child issues.

Practical 37: k-Nearest Neighbors (kNN) classifier

Aim: To implement kNN, split the data and display test accuracy.

Procedure (step by step)

1. Load Iris and split 70 / 30.

2. Scale features with StandardScaler (fit on training data only), because kNN uses distances.

3. Train KNeighborsClassifier for several k values.

4. Print the test accuracy for each k and choose the best k.

Python program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

scaler = StandardScaler().fit(X_train)          # scale using training data only
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

for k in (1, 3, 5, 7):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    print(f"k = {k}: test accuracy = {accuracy_score(y_test, knn.predict(X_test)):.4f}")
```

Output

```
k = 1: test accuracy = 0.9333
k = 3: test accuracy = 0.9111
k = 5: test accuracy = 0.9111
k = 7: test accuracy = 0.9333
```

Practical 38: Customize a workflow by adding statuses and transitions

Aim: To edit an existing workflow and attach it to a project.

Steps in Jira

1. Open Project settings > Workflow (or Jira settings > Workflows) and click Edit on the workflow.

2. Add new statuses such as 'In Review' and 'Blocked'.

3. Add transitions: In Progress -> In Review, In Review -> Done, In Review -> In Progress (Rework), any -> Blocked.

4. Optionally add a rule: only the Project Lead can move to Done.

5. Click Publish. For company-managed projects update the Workflow scheme and click Associate; choose to migrate existing issues.

6. Check the board shows the new columns and test each transition.

Practical 39: Multiclass classification with Logistic Regression

Aim: To perform multiclass classification and display accuracy.

Procedure (step by step)

1. Load the Wine dataset (178 samples, 13 features, 3 classes).

2. Split 75 / 25 (stratified) and scale the features.

3. Train LogisticRegression (multinomial / softmax for 3 classes).

4. Predict and print accuracy and the classification report.

Python program

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

wine = load_wine()                              # 3 classes of wine
X_train, X_test, y_train, y_test = train_test_split(
    wine.data, wine.target, test_size=0.25, random_state=42, stratify=wine.target)

scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

clf = LogisticRegression(max_iter=1000)         # handles multiclass automatically
clf.fit(X_train, y_train)
pred = clf.predict(X_test)

print("Classes:", [str(c) for c in wine.target_names])
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print(classification_report(y_test, pred, target_names=wine.target_names))
```

Output

```
Classes: ['class_0', 'class_1', 'class_2']
Accuracy: 1.0
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        15
     class_1       1.00      1.00      1.00        18
     class_2       1.00      1.00      1.00        12

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Practical 40: Pie Chart / Issue Statistics report on a custom dashboard

Aim: To generate a Pie Chart or Issue Statistics report and add it to a custom dashboard.

Steps in Jira

1. Dashboards > Create dashboard and name it.

2. Add gadget > Pie Chart. Choose Project or Saved filter and Statistic type = Status (or Priority / Assignee).

3. Add gadget > Issue Statistics with the same filter and statistic = Assignee.

4. Save, then arrange gadgets and click Done.

5. The Pie chart shows the share of each status; Issue Statistics shows counts and percentages in a table.

Practical 41: Class imbalance and possible bias in a public dataset

Aim: To analyse a public dataset for class imbalance and bias, show class counts and discuss limitations.

Procedure (step by step)

1. Load a public dataset (Breast Cancer Wisconsin from UCI, via scikit-learn).

2. Count samples per class with value_counts() and convert to percentages.

3. Compute the imbalance ratio and the majority-class baseline accuracy.

4. Compare feature statistics per class to look for differences.

5. Write the limitations (see below).

Python program

```
import pandas as pd
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer(as_frame=True)         # public UCI/Wisconsin dataset
y = data.target.map({0: 'malignant', 1: 'benign'})

counts = y.value_counts()
print("Class counts:")
print(counts)
print()
print("Class percentages (%):")
print((counts / len(y) * 100).round(2))
print()
print("Imbalance ratio (majority/minority): %.2f" % (counts.max() / counts.min()))
print("Majority-class baseline accuracy   : %.2f%%" % (counts.max() / len(y) * 100))

# Quick look at possible measurement bias: feature averages per class
print()
print(data.frame.groupby(y)[['mean radius', 'mean texture']].mean().round(2))
```

Output

```
Class counts:
target
benign       357
malignant    212
Name: count, dtype: int64

Class percentages (%):
target
benign       62.74
malignant    37.26
Name: count, dtype: float64

Imbalance ratio (majority/minority): 1.68
Majority-class baseline accuracy   : 62.74%

           mean radius  mean texture
target
benign           12.15         17.91
malignant        17.46         21.60
```

Class imbalance and bias: discussion

- Imbalance: 357 benign (62.7%) versus 212 malignant (37.3%), ratio 1.68. This is mild; a model that always predicts 'benign' already scores 62.7%, so accuracy alone is misleading. Use precision, recall, F1, ROC-AUC, class weights or resampling (SMOTE).
- Sampling bias: data comes from one hospital / one time period, so it may not represent other populations, ages or equipment.
- Missing demographics: no information on age, ethnicity or region, so fairness across groups cannot be checked.
- Label noise: labels come from biopsy diagnosis, which can contain human error.
- Small size: 569 samples give high variance; results should be confirmed with cross-validation and external data.
- Always check imbalance in sensitive attributes (gender, race) when using real-world data about people.

Practical 42: Create, test and enable a Jira Automation rule

Aim: To create an automation rule, test it and enable it after verifying trigger, condition and action.

Steps in Jira

1. Project settings > Automation > Create rule.

2. Select a trigger (for example: Issue created).

3. Add a condition (for example: Issue type = Bug).

4. Add an action (for example: Assign issue to Project Lead, then Send email).

5. Test: create a Bug and check the rule's Audit log. Each component should show a green tick.

6. Verify the order Trigger -> Condition -> Action works as expected, then click Turn it on.

Practical 43: Naive Bayes with train / test split shown

Aim: To implement Naive Bayes on a small dataset and show how it is divided into training and testing sets.

Procedure (step by step)

1. Create the classic 14-row 'Play Tennis' weather dataset (Outlook, Temp, Humidity, Windy, Play).

2. Encode categorical text columns into integers with OrdinalEncoder.

3. Split with train_test_split (test_size = 0.3, stratified) and print the number of rows and class counts in each part.

4. Train CategoricalNB on the training rows.

5. Predict the test rows and print accuracy. With only 5 test rows, each mistake costs 20% accuracy, so results vary with the split.

Python program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder
from sklearn.metrics import accuracy_score

df = pd.DataFrame({
 'Outlook': ['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny','Sunny','Rain','Sunny','Overcast','Overca
 'Temp':    ['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild','Mild','Mild','Hot','Mild'],
 'Humidity':['High','High','High','High','Normal','Normal','Normal','High','Normal','Normal','Normal','High','Normal','Hi
 'Windy':   ['No','Yes','No','No','No','Yes','Yes','No','No','No','Yes','Yes','No','Yes'],
 'Play':    ['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','No']})

X = OrdinalEncoder().fit_transform(df[['Outlook', 'Temp', 'Humidity', 'Windy']]).astype(int)
y = df['Play']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=7, stratify=y)

print("Total rows    :", len(df))
print("Training rows :", len(X_train), "(%.0f%%)" % (len(X_train) / len(df) * 100))
```

```
print("Testing rows  :", len(X_test), "(%.0f%%)" % (len(X_test) / len(df) * 100))
print("Train class counts:", y_train.value_counts().to_dict())
print("Test  class counts:", y_test.value_counts().to_dict())

nb = CategoricalNB()
nb.fit(X_train, y_train)
pred = nb.predict(X_test)
print("Predicted:", pred.tolist())
print("Actual   :", y_test.tolist())
print("Accuracy :", round(accuracy_score(y_test, pred), 4))
```

Output

```
Total rows    : 14
Training rows : 9 (64%)
Testing rows  : 5 (36%)
Train class counts: {'Yes': 6, 'No': 3}
Test  class counts: {'Yes': 3, 'No': 2}
Predicted: ['Yes', 'No', 'Yes', 'Yes', 'Yes']
Actual   : ['No', 'No', 'Yes', 'Yes', 'Yes']
Accuracy : 0.8
```

Practical 44: Configure project roles and review role permissions

Aim: To configure project roles and review the permissions granted to each role.

Steps in Jira

1. Jira settings > System > Project roles to see the default roles (Administrators, Developers, Users) and add new ones.
2. Open the project > Project settings > People and assign users to roles.
3. Open Project settings > Permissions to see which roles hold which permissions.
4. Review: Administrators = Administer Projects; Developers = create, edit, transition and assign issues; Users = browse and comment.
5. Use Permission helper to confirm what a particular user can do.
6. Document the role-permission matrix as a table in your practical file.